

Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

División de Ingeniería

Ingeniería en Ciencias y Sistemas

Compiladores 2

Ing Mario Tobar



MANUAL TÉCNICO

Diego Alessandro Gonzalez Bravo / 202230939

Quetzaltenango, 26 de septiembre

2026

Descripción general

El sistema es un compilador con interfaz gráfica que recibe un proyecto escrito en tres lenguajes:

Lenguaje	Extensión	Rol dentro del proyecto
Pig Latin	.pig	Archivo principal. Declara variables globales, funciones propias y la sección MAIOR> con el programa a ejecutar. Importa los demás archivos.
Y?	.y	Define estructuras (%estructuras) y funciones (%funciones). Los bloques se delimitan por indentación, como en Python.
Zetariano	.z	Define una clase por archivo, con atributos, constructores y métodos. El archivo debe llamarse igual que la clase.

A partir del archivo .pig el compilador:

1. Analiza léxica y sintácticamente cada archivo, siguiendo las importaciones.
2. Une todos los módulos en un solo AST y lo valida semánticamente.
3. Genera código de tres direcciones (C3D) en forma de cuartetos.
4. Traduce las cuartetos a un archivo C que se compila con gcc.

Los errores de cualquier fase se muestran en una tabla con su tipo, descripción, línea, columna y archivo.

Requisitos y tecnologías

Herramienta	Versión	Uso
Java (JDK)	21	Lenguaje de implementación. El proyecto usa expresiones switch, instanceof con patrones, record y bloques de texto.
Apache Maven	3.9 o superior	Construcción, dependencias y generación de los analizadores.
ANTLR 4	4.13.1	Generador de lexers y parsers (antlr4-maven-plugin y antlr4-runtime).
Java Swing	incluido en el JDK	Interfaz gráfica. No se usan librerías visuales externas.
GCC	cualquier versión reciente	Compilar el archivo .c generado. No lo usa el compilador en sí.

La única dependencia de ejecución es org.antlr:antlr4-runtime:4.13.1.

Compilación y ejecución del proyecto

Generar los analizadores

Las gramáticas están en `src/main/antlr4/`. El plugin de ANTLR genera los lexers, parsers, visitors y listeners en `src/main/java/Backend/antlr/` con el paquete `Backend antlr`. Se regeneran en la fase `generate-sources` de Maven, que también se ejecuta al compilar:

```
mvn generate-sources
```

Los archivos de `Backend/antlr/` no se editan a mano: cualquier cambio se pierde al regenerar

Construir y ejecutar

```
mvn clean package
```

```
java -jar target/Proyecto1-compi-1.0-SNAPSHOT-jar-with-dependencies.jar
```

`maven-assembly-plugin` genera un jar ejecutable que incluye el runtime de ANTLR y tiene `Main.Main` como clase principal. Desde un IDE basta con ejecutar la clase `Main.Main`.

Compilar el código C generado

Después de guardar el código desde la pestaña **CODIGO GENERADO** (por defecto `salida.c`):

```
gcc salida.c -o salida -lm
```

```
./salida
```

Fase 1: análisis léxico, sintáctico y módulos Gramáticas

Las tres gramáticas se escribieron sin código embebido (sin acciones ni predicados en Java). Toda la lógica vive en las clases Java, lo que permite regenerar los analizadores sin perder cambios.

Gramática	Regla inicial	Estructura del programa	Comentarios
pigLatin.g4	programa	imports, VARIABLES> (opcional), MUNERA> (opcional), MAIOR> ... FINIS (obligatoria)	// línea y ## bloque ##
Y.g4	programa	%estructuras (opcional) y %funciones (obligatoria)	// línea y /* bloque */
Zetariano.g4	programa	Una sola definicionClase	// línea y /* bloque */

Los tres lenguajes distinguen mayúsculas de minúsculas. En Pig Latin, finis cierra bloques (si, dum, per, funciones) y FINIS cierra la sección MAIOR>.

Tipos primitivos de cada lenguaje y su categoría interna (Type.TypeCategory):

Categoría	Pig Latin	Y?	Zetariano
INT	numerus	entero	int
FLOAT	decimalis	flotante	double
CHAR	littera	caracter	char
BOOLEAN	bool (verum, falsus)	bool (verdadero, falso)	boolean (true, false)
STRING	textum	cadena	String
STRUCT	estructura importada de un .y	estructura	no aplica
CLASS	clase importada con novus	no aplica	class
ARRAY	series nombre[n] : tipo	tipo nombre[n]	tipo[], new tipo[n]

Además existen VOID (funciones sin retorno), NULL (null de Zetariano) y ERROR (tipo interno que reciben las expresiones inválidas para que el análisis continúe).

Instrucciones de entrada y salida:

Operación	Pig Latin	Y?	Zetariano
Imprimir con salto de línea	>> a >> b;	imprimir(a)	println(a)
Imprimir sin salto	no aplica	no aplica	print(a)
Leer	variable <<	leer()	readln()

En Pig Latin cada valor impreso lleva su propio >>. Si una línea nueva empieza con >>, PigLatinAstVisitor la trata como una impresión separada.

Indentación en Y? (YIndentTokenSource)

Y? delimita los bloques por indentación. ANTLR no genera esos tokens por sí mismo, así que la gramática declara tokens { INDENT, DEDENT } y la clase YIndentTokenSource se coloca entre el lexer y el parser:

YLexer -> YIndentTokenSource -> CommonTokenStream -> YParser

Funcionamiento:

- El lexer produce NEWLINE con los espacios de la línea siguiente incluidos. Una tabulación cuenta como 4 espacios (ESPACIOS_POR_TABULACION).
- Una Pila<Integer> guarda los niveles de indentación abiertos; inicia con 0.
- Si la nueva línea tiene más indentación que la cima, se apila el nivel y se emite INDENT.
- Si tiene menos, se desapilan niveles y se emite un DEDENT por cada uno.
- Las líneas en blanco consecutivas se unen en un solo salto.
- Al llegar al final del archivo se cierran todos los bloques abiertos y se agrega un salto final si faltaba.

- Los tokens pendientes se guardan en una Cola<Token> y se entregan uno por uno al parser.

La gramática también acepta llaves { } como alternativa a la indentación.

Construcción del AST (AstBuilder y visitors)

AstBuilder.analizarArchivo(Path) detecta el lenguaje por la extensión y ejecuta:

1. El lexer del lenguaje (más YIndentTokenSource en Y?).
2. El parser, comenzando en la regla programa.
3. Un ListenerErroresAntlr conectado al lexer y al parser, en lugar de los listeners de consola de ANTLR.
4. El visitor correspondiente, que recorre el árbol de ANTLR y devuelve un ProgramNode:
 - PigLatinAstVisitor (extiende pigLatinBaseVisitor<Node>)
 - YAstVisitor (extiende YBaseVisitor<Node>)
 - ZetarianoAstVisitor (extiende ZetarianoBaseVisitor<Node>)

Detalles relevantes:

- Literales resuelve las secuencias de escape (\n, \t, \", \', \\, etc.) de cadenas y caracteres para los tres lenguajes.
- Cada declaración (StructDecl, ClassDecl, FunctionDecl) queda marcada con su archivo de origen (archivoOrigen). Así, un error semántico dentro de un módulo importado se reporta con el archivo correcto.
- El resultado de cada archivo es un ResultadoAnálisis con el AST, sus errores, el lenguaje y el nombre del archivo.

Coloreado de sintaxis

El coloreado usa los mismos lexers de ANTLR que el compilador, sin librerías externas:

- LenguajeColoreado (enum PIG_LATIN, Y, ZETARIANO) elige el lexer según la extensión del archivo y clasifica cada tipo de token en una CategoriaToken.
- CategoriaToken (enum) define el color, la negrita y la cursiva de cada categoría: palabra reservada, tipo de dato, sección, función del sistema, función, identificador, cadena, número, constante, operador, delimitador y comentario.

- ColoreadorSintaxis.analizar recorre los tokens y produce una lista de Segmento (inicio, fin, categoria):
 - Un identificador seguido de (se colorear como función.
 - Un identificador después de novus, new, class o estructura se colorear como tipo.
 - Las gramáticas descartan los comentarios (-> skip), así que el texto entre tokens que no es espacio en blanco se colorear como comentario.
 - Los caracteres que el lexer no reconoce se dejan sin color: el coloreado no marca errores.
 - PosicionesTexto convierte posiciones de ANTLR (puntos de código) a posiciones de Java (caracteres), para textos con emojis u otros caracteres fuera del plano básico.
- ColoreadorSintaxis.aplicar pinta los segmentos en el StyledDocument del editor.
- PestaniaEditor recolorea 200 ms después de la última tecla (un Timer de Swing). Los cambios de estilo no marcan el archivo como modificado.
- Después de compilar, Inicio.colorearSegunCompilacion reúne las clases y estructuras de todos los módulos y las pasa a cada editor (aplicarTiposDelProyecto). Así, un tipo definido en otro archivo también se colorear como tipo de dato.

12. Manejo de errores

Clase	Función
TipoError	Enum LEXICO, SINTACTICO, SEMANTICO.
ErrorCompilacion	Tipo, mensaje, línea, columna, archivo y lexema del error.
GestorErrores	Acumula los errores. Guarda el archivo actual para asignarlo a cada error, y permite unir los errores de varios archivos (agregarTodos).
ListenerErroresAntlr	Reemplaza los listeners de consola de ANTLR. Si el error trae un token (error del parser), se registra como sintáctico ; si no (el lexer no reconoció un carácter), como léxico .

Los mensajes léxicos y sintácticos son los que produce ANTLR (por ejemplo missing 'finis' at 'FINIS'). Los semánticos los redacta el analizador (por ejemplo ID NO DECLARADO: x).

El parser de ANTLR se recupera de los errores y sigue analizando, de modo que en una sola compilación se reportan varios errores sintácticos. El análisis semántico tampoco se detiene en el primer error: la expresión inválida recibe el tipo ERROR y el recorrido continúa con el resto del programa.

En tiempo de ejecución, el programa C solo valida la división y el módulo entre cero.

Estructuras de datos propias

El paquete Backend.estructuras implementa estructuras genéricas enlazadas, todas iterables:

Clase	Operaciones	Uso en el compilador
Pila<T>	apilar, desapilar, cima, cimaONulo, estaVacia, tamaño	Niveles de indentación (YIndentTokenSource), ámbitos abiertos (TablaSimbolos), marcos activos (GeneradorC3D) y etiquetas de break/continue (ControladorC3D).
Cola<T>	encolar, desencolar, frente, frenteONulo, estaVacia, tamaño	Tokens INDENT/DEDENT pendientes (YIndentTokenSource).
ListaEnlazada<T>	agregar, obtener, establecer, contiene, tamaño	Disponible como estructura auxiliar.